

Primer proyecto

El primer proyecto, tiene como objetivo principal, realizar un esqueleto de un proyecto funcional en Free-RTOS, el mismo luego podrá ser usado como template para próximos proyectos.

Implementaremos entonces una simple tarea que realizará un parpadeo en el led de la placa BluePill

Para esta tarea vamos a tener que:

1. Crear una carpeta para nuestro proyecto junto con una estructura interna de subcarpetas.
2. Bajar la librería CMSIS y copiar los archivos necesarios a la carpeta de nuestro proyecto.
3. Bajar la librería FreeRTOS y copiar los archivos necesarios a la carpeta de nuestro proyecto.
4. Modificar algunos de los archivos de CMSIS y FreeRTOS para vincular los proyectos entre sí.
5. Generar desde cero un serie de archivos necesario para el funcionamiento de la aplicación
6. Compilar y grabar en nuestra placa BluePill

Paso 1 - Estructura de subcarpetas

Carpeta principal de nuestra aplicación y árbol de subcarpetas

- **rtos-blink** → nombre de la carpeta / proyecto
 - **FreeRTOS** → carpeta con todos los archivos del RTOS
 - **CMSIS** → carpeta con todos los archivos de la librerías CMSIS
 - **src** → carpeta con los archivos fuentes exclusivos de la aplicación (main.c por ejemplo)
 - **ld** → carpeta que contiene el linker script

Paso 2 - Common Microcontroller Software Interface Standard (CMSIS)

CMSIS: es una librerías promovida por ARM, que defines registros y funciones para el manejo del procesador, esto permite además estandarizar estas funciones para poder migrar fácilmente entre distintos Cortex-M

Estructura del proyecto CMSIS

- **CMSIS-Core**
 - Define registros y funciones del procesador (NVIC, SysTick, control de interrupciones, etc.).

- Archivos fundamentales para iniciar una aplicación como: `core_cm3.h`, `system_stm32f1xx.c`.
- Permite que tu código sea portable entre diferentes fabricantes (ST, NXP, TI, etc.).
- **CMSIS-Driver**
 - Una API estándar para periféricos comunes (I²C, SPI, USART, etc.).
 - Posee Template para que cada fabricante implemente la API para su hardware.
- **CMSIS-DSP**
 - Librería optimizada de procesamiento digital de señales (FFT, filtros, PID, etc.).
- **CMSIS-RTOS**
 - API estándar para sistemas operativos en tiempo real (ej: FreeRTOS, RTX).
 - Permite escribir aplicaciones independientes del RTOS usado.
 - No la vamos a usar en nuestro caso para no complicar aun mas la generación del proyecto.

En esta librería, solo encontraremos las implementaciones generales que son relativamente independiente del hardware, luego tendremos muchos prototipos.

CMSIS está pensado para usarlo como base y a partir de allí, crear la CMSIS particular para cada microcontrolador y fabricante.

De esta librería por ahora, solo necesitaremos algunas cabeceras que serán requeridas en nuestro proyecto.

La librería se puede bajar desde <https://www.arm.com/technologies/cmsis>

De allí vamos al botón “download”, esto nos abre el proyecto en github y podemos bajar el .zip del proyecto completo o ir al “code” (equina superior izquierda) y bajar solo los archivos necesarios, en cualquier caso nos dirigimos a la carpeta siguiente:

CMSIS_6/CMSIS/Core/Include

y bajamos o copiamos los siguientes archivos a nuestra carpeta **CMSIS** de la aplicación.

- **cmsis_compiler.h**
- **cmsis_gcc.h**
- **cmsis_version.h**
- **core_cm3.h**

El resto de los archivos necesarios los encontramos en la implementación de esta librería para el microcontrolador que posee la BluePill, denominado **STM32CubeF1** ubicado en la siguiente dirección

<https://github.com/STMicroelectronics/cmsis-device-f1>

De allí bajamos los siguientes archivos nuevamente a nuestra carpeta **CMSIS** del proyecto

- **system_stm32f1xx.c** y **system_stm32f1xx.h** de cmsis-device-f1/Source/Templates
Este archivo en C incluye la función **SystemInit()**, encargada de inicializar el reloj, configurar la memoria Flash y otros parámetros básicos. Por defecto, esta función no está implementada, por lo que debemos escribirla manualmente. También contiene la función **SystemCoreClockUpdate()**, la cual debe ser llamada desde el **main()** de nuestro proyecto para actualizar correctamente la variable global **SystemCoreClock**, ubicada en este mismo archivo, que almacena la frecuencia de la CPU.
- **startup_stm32f102xb.s** de cmsis-device-f1/Source/Templates/gcc
Escrito en ensamblador, en este archivo se declara los vectores de interrupción, el primero de ellos es el Reset, interrupción que se llama al momento de encender el mico, este vector es atendido por la función **Reset_Handler**, ubicada en el mismo archivo.
La función **Reset_Handler** realiza los siguientes procesos:
 - Llama a la función **SystemInit()**
 - Copia .data desde Flash a RAM, es decir las variables globales preinicializadas
Ejemplo `int contador = 5;`
 - Inicializa .bss en cero, es decir las variables globales no inicializadas
Ejemplo `int banderas;`
 - Llama a la función **__libc_init_array()** librería interna de la newlib que realiza diferentes inicializaciones para el correcto funcionamiento de las funciones de newlib
 - llama al **main()**
- **stm32f1xx.h**, **system_stm32f1xx.h** y **stm32f103xb.h** de cmsis-device-f1/Include

Luego en la carpeta de nuestro proyecto denominada **ld**, copiamos **STM32F103XB_FLASH.ld** de cmsis-device-f1/Source/Templates/gcc/linker.

Este archivo es el linker script, utilizado por el enlazador para recibir las instrucciones de como se agrega cada archivo objeto al binario final.

El objetivo de este script es indicarle al enlazador cómo se deben mapear las secciones de los archivos de entrada en el archivo de salida. Además, permite al enlazador ejecutar otras operaciones según los comandos realizados en lenguaje de script.

El script más simple consiste solamente en un comando **SECCIONES** que describe la disposición de las distintas secciones en el archivo de salida. Por ejemplo, podría indicar que la sección de texto se cargue en la dirección `0x0000 0000` y la sección de datos en la dirección `0x2000 0000`.

A continuación se describe el linker script **STM32F103XB_FLASH.ld** predefinido que se bajó del proyecto **STM32CubeF1**

```
/* Entry Point */  
ENTRY(Reset_Handler)  
  
/* Highest address of the user mode stack */  
_estack = 0x20004FFF; /* end of RAM */
```

ENTRY(Reset_Handler) → esta es la primera línea de código script que encontramos (lo demás son comentarios) y detalla el nombre de la primera función que se ejecutará en el sistema, esta función en nuestro caso se encuentra en el **startup_stm32f102xb.s**

_estack = 0x20004FFF → asigna el valor más alto de la memoria RAM a la variable **_estack**, esta variable luego se utiliza en el startup para cargar el MSP o Main Stack Pointer, registro que se encuentra en la primera posición del vector de interrupciones

Tamaños Mínimos de Heap y Stack

```
/* Generate a link error if heap and stack don't fit into RAM */  
_Min_Heap_Size = 0x200; /* required amount of heap */  
_Min_Stack_Size = 0x400; /* required amount of stack */
```

Se definen dos símbolos para establecer tamaños mínimos para el montículo (heap) y la pila (stack), estas variables se utilizan luego en la asignación de memoria RAM, se le suma a las asignaciones para verificar que no exceda la memoria máxima

_Min_Heap_Size = 0x200 Reserva un mínimo de 512 bytes para el heap, que se usa en asignaciones de memoria dinámicas (con malloc).

_Min_Stack_Size = 0x400 Reserva un mínimo de 1024 bytes para el stack, usado para variables locales y llamadas a funciones. Estos valores no definen el tamaño final, sino que son usados más adelante para una comprobación de seguridad.

Definición de las Áreas de Memoria

```
/* Specify the memory areas */  
  
MEMORY  
{  
  FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 128K  
  RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 20K  
}
```

Este bloque es el mapa de la memoria física del microcontrolador.

- FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 128K
Define la memoria Flash, dirección origen, longitud y atributos rx=read (lectura) y execute (ejecución)
- RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
Define la memoria RAM, dirección origen, longitud y atributos xrw=read (lectura), execute (ejecución) y escritura

Definición y Ubicación de las Secciones

El bloque SECTIONS es donde se organiza el contenido del programa.

Es decir donde y como se organizaran en la memoria del microcontrolador cada sección de la aplicación

Tabla de Vectores de Interrupción

```
/* The startup code goes first into FLASH */  
  
.isr_vector :  
{  
    . = ALIGN(4);  
    KEEP(*(.isr_vector)) /* Startup code */  
    . = ALIGN(4);  
} >FLASH
```

se define la seccion .isr_vector y se le indica que debe ir en la FLASH.

Esta sección contiene la tabla de vectores de interrupción. Es fundamental que esté al principio de la Flash, ya que el hardware del microcontrolador espera encontrarla en la dirección 0x08000000.

KEEP(*(.isr_vector)) le ordena al enlazador que no descarte esta sección, incluso si piensa que no se está usando.

Código del Programa y Datos de Solo Lectura

```
/* The program code and other data goes into FLASH */  
.text :  
{  
    . = ALIGN(4);  
    *(.text)           /* .text sections (code) */  
    *(.text*)          /* .text* sections (code) */  
    ...  
    . = ALIGN(4);  
    _etext = .;         /* define a global symbols at end of code */  
} >FLASH  
  
/* Constant data goes into FLASH */  
.rodata :  
{  
    . = ALIGN(4);  
    *(.rodata)          /* .rodata sections (constants, strings, etc.) */  
    *(.rodata*)         /* .rodata* sections (constants, strings, etc.) */
```

```
. = ALIGN(4);  
} >FLASH
```

- `.text`: Agrupa todo el código ejecutable (`.text` y `.text*`) de tu programa y lo coloca en FLASH. El símbolo `_etext` se crea para marcar el final del código. (* luego de `.text` es para agrupar todas las secciones que comiencen con `.text`)
- `.rodata`: Agrupa todos los datos de solo lectura, como constantes (`const`) y cadenas de texto, y también los coloca en FLASH.

Variables Inicializadas (`.data`)

```
/* used by the startup to initialize data */  
_sidata = LOADADDR(.data);  
/* Initialized data sections goes into RAM, load LMA copy after code */  
.data :  
{  
    . = ALIGN(4);  
    _sdata = .;          /* create a global symbol at data start */  
    *(.data)              /* .data sections */  
    *(.data*)             /* .data* sections */  
    . = ALIGN(4);  
    _edata = .;          /* define a global symbol at data end */  
} >RAM AT> FLASH
```

La sección `.data` contiene variables globales y estáticas con un valor inicial (ej: `int valor = 100;`).

`>RAM AT> FLASH`: Esta directiva indica que la sección se ubicará en la RAM durante la ejecución, pero sus valores iniciales se almacenarán en la FLASH.

El startup deberá copiar los valores de inicio localizado en la FLASH a la posición correspondiente de la RAM, para este trabajo necesita los siguientes datos:

- `_sidata`: La dirección en FLASH donde comienzan los valores iniciales.
- `_sdata`: La dirección en RAM donde se deben copiar esos valores.
- `_edata`: La dirección en RAM donde termina la sección.

Al arrancar, el `Reset_Handler` copia el bloque de memoria desde `_sidata` (en Flash) a `_sdata` (en RAM), con una longitud de `_edata - _sdata`.

Variables no Inicializadas (`.bss`)

Esta sección es similar a la anterior, la diferencia es que no hay valores de inicio, así que no hay utilización de FLASH, y los parámetros a enviar al startup serán inicio y fin del bloque para poder poner a 0 toda esta sección de RAM antes de arrancar.

Resumen

En la carpeta CMSIS de nuestro proyecto realizamos las siguientes copias

De CMSIS_6/CMSIS/Core/Include (proyecto CMSIS principal)

- cmsis_compiler.h
- cmsis_gcc.h
- cmsis_version.h
- core_cm3.h

De cmsis-device-f1/Source/Templates (proyecto STM32CubeF1)

- system_stm32f1xx.c
- system_stm32f1xx.h

De cmsis-device-f1/Source/Templates/gcc

- startup_stm32f102xb.s

De cmsis-device-f1/Include

- stm32f1xx.h
- system_stm32f1xx.h
- stm32f103xb.h

En la carpeta ld de nuestro proyecto copiamos

de cmsis-device-f1/Source/Templates/gcc/linker

- STM32F103XB_FLASH.ld

FreeRTOS

Es un sistema operativo de tiempo real utilizado en microcontroladores y pequeños microprocesadores.

El proyecto se puede bajar desde <https://www.freertos.org/>

De allí está la opción para bajar el paquete completo que luego se puede ubicar en una carpeta separada de nuestro proyecto.

Esto se realiza para luego copiar solo los fuentes necesarios en nuestro proyecto y de esa manera tener un control mas preciso de que estamos usando realmente de los cientos de archivos que tiene el proyecto FreeRTOS

Los archivos necesarios para el proyecto se encuentra en la siguiente carpeta

FreeRTOSxx/FreeRTOS-LTS/FreeRTOS/FreeRTOS-Kernel

donde FreeRTOSxx representa la versión del paquete que se bajó de la página

Los archivos de esta carpeta y de las subcarpetas examples e include, se pueden copiar completas a la carpeta **FreeRTOS** de nuestro proyecto.

En la carpeta portable, se encuentra los port para las diferentes plataformas de desarrollo y arquitecturas, en nuestro caso solo necesitamos las siguientes carpetas

- ../FreeRTOS-Kernel/portable/GCC/ARM_CM3 carpeta específica con el port para ARM Cortex-M3 utilizando GCC como plataforma
- ../FreeRTOS-Kernel/portable/MemMang carpeta con diferente implementaciones para manejo de memoria dinámica requerida por RTOS

Copiamos entonces ambas carpetas al FreeRTOS de nuestro proyecto, procurando mantener la estructura de carpetas, es decir que nos quede dentro del FreeRTOS de nuestro proyecto una carpeta portable/GCC/ARM_CM3 y portable/MemMang

Resumen

En la carpeta FreeRTOS de nuestro proyecto realizamos las siguientes copias

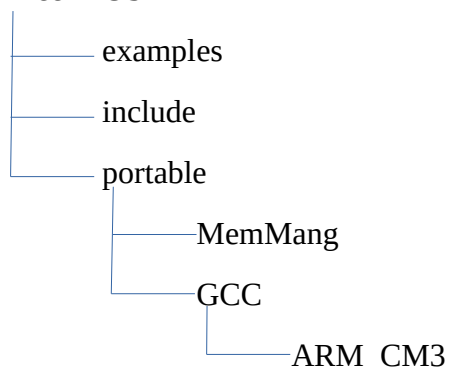
Los archivos de la carpeta FreeRTOSxx/FreeRTOS-LTS/FreeRTOS/FreeRTOS-Kernel

y las siguientes carpetas

- ../FreeRTOS-Kernel/examples
- ../FreeRTOS-Kernel/include
- ../FreeRTOS-Kernel/portable/GCC/ARM_CM3
- ../FreeRTOS-Kernel/portable/MemMang

Tendremos entonces en nuestra carpeta FreeRTOS, los archivos de la carpeta FreeRTOS-Kernel y la siguiente estructura de carpetas

FreeRTOS



Archivos a modificar

Una vez cargado los archivos en nuestro proyecto, debemos realizar unas pequeñas modificaciones para poder enlazar a nuestro proyecto basado en CMSIS con el FreeRTOS.

Los archivos a modificar serán los siguientes

- 1) **system_stm32f1xx.c**, este archivo de CMSIS, posee la función de inicialización SystemInit(), esta función en la versión original de CMSIS no hace mucho, para el caso nuestro es importante que modifique el clock del micro a 72 MHz para que funcione correctamente el FreeRTOS, de paso además podemos configurar la unidad Flash entre otras cosas, abrimos entonces este archivo y en la función SystemInit() copiamos lo siguiente

```
void SystemInit(void)
{
    /* 1. Habilitar HSE */
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY));

    /* 2. Configurar Flash: Prefetch + 2 wait states */
    FLASH->ACR |= FLASH_ACR_PRFTBE;
    FLASH->ACR &= ~FLASH_ACR_LATENCY;
    FLASH->ACR |= FLASH_ACR_LATENCY_2;

    /* 3. Configurar prescalers */
    RCC->CFGR |= RCC_CFGR_HPRE_DIV1;    // AHB = SYSCLK /1
    RCC->CFGR |= RCC_CFGR_PPRE1_DIV2;  // APB1 = SYSCLK /2 (36 MHz máx)
    RCC->CFGR |= RCC_CFGR_PPRE2_DIV1;  // APB2 = SYSCLK /1

    /* 4. Configurar PLL (HSE * 9 = 72 MHz) */
    RCC->CFGR &= ~(RCC_CFGR_PLLSRC | RCC_CFGR_PLLXTPRE | RCC_CFGR_PLLMULL);
    RCC->CFGR |= (RCC_CFGR_PLLSRC | RCC_CFGR_PLLMULL9);

    /* 5. Encender PLL y esperar */
    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY));

    /* 6. Cambiar SYSCLK a PLL */
```

```
RCC->CFGR &= ~RCC_CFGR_SW;
RCC->CFGR |= RCC_CFGR_SW_PLL;
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);

/* 7. Actualizar variable global */
SystemCoreClock = 72000000;
}
```

- 2) **port.c**, este archivo contiene las funciones que posee el RTOS para vincularse con el hardware, por esta razón se encuentra en una carpeta que define el hardware que tenemos, en nuestro caso ARM_CM3 (ARM Cortex-M3), aquí se implementan 3 funciones que atienden a 3 interrupciones del microcontrolador, en nuestro caso y al final del archivo, añadimos las siguientes líneas.

```
void SVC_Handler(void) __attribute__((alias("vPortSVCHandler")));
void PendSV_Handler(void) __attribute__((alias("xPortPendSVHandler")));
void SysTick_Handler(void) __attribute__((alias("xPortSysTickHandler")));
```

Estas líneas reemplazan los nombre originales de las funciones que atendían a los vectores según el estándar de CMSIS, por las funciones que provee FreeRTOS.

Archivos a Agregar

Los archivos que deberemos generar desde cero serán los siguientes

- **main.c** donde se encuentra la declaración de una única tarea que realizará un parpadeo en el único led disponible en la BluePill, inicializará el hardware necesario y finalmente lanzará el scheduler del RTOS
- **Makefile** que controlará la compilación y enlace de nuestro proyecto
- **syscalls.c** este es un caso especial y no es requerido por RTOS sino por la librería newlib, esta librería implementa funciones como printf(), scanf(), malloc(), etc. Estas funciones internamente llaman a diferentes funciones que deben ser implementadas según la configuración que tengamos y para esto existe syscalls.

En **syscalls.c** implementaremos las siguiente funciones

```
_write() → enviar datos a stdout
_read() → leer de stdin
_close(), _fstat(), _lseek(), etc.
```

esto permite por ejemplo redirigir el stdout o stdin a una UART, teclado matricial o un display.

Makefile

Un **Makefile** es un archivo de que presenta las reglas para automatizar el proceso de compilación y construcción de un programa.

En una aplicaciones embebidas, el **Makefile** se encarga de:

1. Definir los archivos fuente (.c, .s) de nuestra aplicación.
2. Configurar al compilador (gcc)
3. Controlar el linkeo asignando el linker script y la lista de .o requeridos para generar la aplicación.
4. Generar los archivos finales (.elf .bin .hex) y archivos de informes como el .map
5. Administrar dependencias: recompilar aquellos archivos .c o .s que fueron modificados o se modificaron sus dependencias (.h)
6. Automatizar algunos servicios extras como:
 - `make clean` → para borra objetos y binarios.
 - `make flash` → para programa el micro directamente con `openocd`.

Detalle del Makefile ejemplo para nuestra primera aplicación en Free-RTOS

1. Sección de Variables

1. Toolchain

```
PREFIX    = arm-none-eabi-  
CC        = $(PREFIX)gcc  
OBJCOPY   = $(PREFIX)objcopy
```

Define variables con las herramientas utilizadas para compilar, enlazar, etc

- PREFIX: prefijo del toolchain ARM.
- CC: compilador C (usado también para enlazar y ensamblar)
- OBJCOPY: convertir .elf en .bin o .hex.

2. Banderas de compilación (CFLAGS)

La variable CFLAGS contiene todas las banderas requeridas para la compilación de cada .c

- `-mcpu=cortex-m3` → especifica la CPU destino (Cortex-M3).

- -g → agrega información de depuración para luego poder realizar un debug.
- -I<dir> → incluye todo los directorios donde se encuentra los .h
- -DSTM32F103xB → define una constante STM32F103xB con valor 1 para que CMSIS pueda identificar al micro que se está usando.
- -Wall → advertencias comunes.
- -ffunction-sections / -fdata-sections → define una sección propia para cada función o variable, estas banderas se usan en conjunto con el -gc-sections del linker.
- -MMD -MP → genera los archivos .d donde se detallan las dependencias de cada .c

3. Flags de linkeo (LDFLAGS)

La variable LDFLAGS contiene las banderas requerida para el enlace (linker)

- -T ld/STM32F103XB_FLASH.ld → script para el linkeo.
- -nostartfiles → evita que el enlazador los archivos por defecto que tiene para arrancar nuestra aplicación, en su lugar se incluye startup_stm32f103xb.s que es el archivo que imponemos en nuestra aplicación para definir los vectores de interrupción y lo necesario para arrancar el microcontrolador estándar.
- -Wl,--gc-sections → elimina secciones no usadas (utilizada en conjunto con -ffunction-sections / -fdata-sections permite borrar las funciones no utilizadas y evitar ocupar espacio en el binario de salida).
- -Wl,-Map=app.map → genera mapa de memoria.

Nota: Las dos últimas banderas o directivas requieren anteponer **-Wl,**, lo que indica que son opciones que solo reconoce el linker. En nuestra aplicación estamos usamos el GCC para enlazar, pero en realidad lo que hace GCC es invocar a ld (enlazador de GNU), pasando toda la información que le proporcionamos como parámetros. La mayoría de las banderas son reconocidas por GCC como opciones de linker y se le transmiten automáticamente. Sin embargo, algunas como **-Map** y **--gc-sections**, no son reconocidas directamente por GCC. Para evitar errores, les anteponemos **-Wl,**, indicando a GCC que las transfiera tal cual al linker.

4. Fuentes y objetos

- SRCS = lista de todos los archivos fuente que pertenecen a la aplicación (.c, .s), es la lista que modificaremos sumando o quitando archivos durante la evolución de nuestra aplicaciones.
- OBJS = lista de objetos (.o) generados en base al listado de SRCS.
- DEPS = dependencias automáticas (.d).

2. Reglas principales

Esta reglas permiten generar el archivo binario final (app.elf) y archivos binario de otro formato (app.hex y app.bin)

1. **all: app.elf app.bin app.hex**

Es la regla principal, cuando uno escribe make o make all (es lo mismo), make busca la regla para el all en el Makefile y encuentra esta linea, los : indica de que depende all, en este caso son 3 archivos, cada archivo además tendrá su dependencia (mas abajo) make, intentará entonces cumplir la dependencia de este listado de archivos

2. **app.elf: \$(OBJS)**
\$(CC) \$(CFLAGS) \$^ -o \$@ \$(LDFLAGS)

app.elf es el archivo final de salida, utilizado por el openocd para grabar el binario en el microcontrolados.

Aquí app.elf depende de una lista de .o definida en la variable \$(CFLAGS), make buscará la regla de cada .o en la lista, luego de cumplirse esa regla, verifica la dependencia de app.elf con el listado de los .o, que es básicamente analizar las fechas de generación de cada uno, si algún .o es mas reciente que app.elf, ejecuta la regla asociada, es decir la linea inferior la cual enlaza el proyecto.

3. **app.bin: app.elf**
\$(OBJCOPY) -O binary \$< \$@

El Archivo app.bin es un binario plano, no es utilizado en nuestro caso, lo mantenemos por si se requiere grabar con el st-link (aplicación propia de ST para grabar), u otro formato de grabación como el directo por USB

Funcionamiento de la regla: Aquí la dependencia de app.bin es solo de app.elf, cumplida la regla de app.elf, si se modificó genera nuevamente el app.bin

4. **app.hex: app.elf**
\$(OBJCOPY) -O ihex \$< \$@

El Archivo app.hex es un binario de formato Intel, no es utilizado en nuestro caso.

Funcionamiento de la regla: idem al anterior.

3. Reglas de compilación
tendremos 2 reglas para generar cada uno de los .o

1. **%.o: %.c**
\$(CC) \$(CFLAGS) -c -o \$@ \$<

Es la regla para generar .o de un .c, utiliza para esto el comodín % de make para que cada archivo .o dependerá de un archivo .c con el mismo nombre y ruta, el comodín evita escribir la dependencia de cada archivo .o, por ejemplo:

src/main.o : src/main.c

CMSIS/system_stm32f1xx.o : CMSIS/system_stm32f1xx.c

...

Al igual que casos anteriores si el .o es anterior al .c se ejecuta la regla que compila y ensambla el .c

2. **%.o: %.s**
\$(CC) -c -mcpu=cortex-m3 -mthumb -o \$@ \$<

Es la regla para generar el .c de un .s, al igual que el caso anterior utiliza el comodín % y ensambla los .s con fecha posterior a su .o

4. Dependencias automáticas

-include \$(DEPS) → incluye dependencias generadas automáticamente.

Permite recompilar solo los archivos afectados cuando cambia un .h.

5. Extras

1. flash: app.elf

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c  
"program app.elf verify reset exit"
```

programa la blue pill con openocd usando app.elf.

clean:

2. rm -f \$(OBSJ) \$(DEPS) app.elf app.map

Elimina binarios y objetos.

main.c

Punto de entrada de nuestra aplicación, contiene las funciones que inicializa el hardware, luego de esto, los pasos siguientes dependerá del tipo de aplicación.

En una aplicación clásica de baremetal, el main generalmente contiene el bigloop o lazo principal de la aplicación donde queda en un bucle infinito.

Para el caso de FreeRTOS, es en el main donde se crean las tareas, para luego dejar paso a la función `vTaskStartScheduler()` donde el FreeRTOS toma el control del microprocesador.

syscall.c

Este archivo como se hablo antes, sirve para implementar las funciones de entrada /salida y otras funciones específicas de hardware, en este caso se inicializan en blanco para luego si es necesario realizar la implementación

```
#include <sys/stat.h>
```

```
#include <errno.h>
```

```
#include <stdint.h>
```

```
void _exit(int status) {
```

```
    (void)status;
```

```
    while (1) {}
```

```
}
```

```
int _close(int file) { return -1; }
```

```
int _fstat(int file, struct stat *st) { st->st_mode = S_IFCHR; return 0; }
```

```
int _isatty(int file) { return 1; }  
int _lseek(int file, int ptr, int dir) { return 0; }  
int _read(int file, char *ptr, int len) { return 0; }  
int _write(int file, char *ptr, int len) { return len; }  
void _init(void) {}
```